DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 2
Manejo de Ficheros
Parte 3 - Jackson

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD2 – Manejo de ficheros

ÍNDICE

15. Librería Jackson
- 15.1.- Mapeo con archivos en formato JSON
- 15.2.- Mapeo con archivos en formato YAML
- 15.3.- Mapeo con archivos en formato CSV
- 15.4.- Mapeo con archivos en formato XML

15. Librería Jackson

Jackson es una librería de Java para procesar datos en formato JSON (parsing y generación).

Principales características y usos:

- **¿Qué hace?**
  - **Serialización**: convierte objetos Java a JSON.
  - **Deserialización**: convierte JSON a objetos Java.
  - **Lectura/escritura** incremental (streaming) y mapeo de árboles (JsonNode).
- **Componentes clave:**
  - **ObjectMapper**: clase principal para convertir entre Java y JSON.
  - **JsonParser/JsonGenerator**: API de streaming para procesar grandes flujos.
  - **JsonNode/Tree Model**: representación de árbol para manipular JSON dinámico.
  - **Anotaciones** (ej.: @JsonProperty, @JsonIgnore, @JsonCreator) para personalizar el mapeo.
- **Ventajas:**
  - Rápida y eficiente.
  - Flexible: soporte para configuraciones, módulos (p. ej. JavaTimeModule) y tipos complejos.
  - Amplio ecosistema (integrado en frameworks como Spring).

Ejemplo

```
ObjectMapper mapper = new ObjectMapper();

// Serializar
String json = mapper.writeValueAsString(miObjeto);

// Deserializar
MiClase obj = mapper.readValue(jsonString, MiClase.class);
```

Además, Jackson es modular y soporta varios formatos además de JSON mediante módulos/proveedores de dataformat. Opciones comunes:

XML	jackson-dataformat-xml	mapper que lee/escribe XML usando XmlMapper
YAML	jackson-dataformat-yaml	basado en SnakeYAML
CSV	jackson-dataformat-csv	mapear filas CSV a POJOs o JsonNode

Para simplificar su uso, el profesor proporcionará una clase de utilidad denominada **JacksonMapper** que contendrá métodos static para realizar el mapeo en los diferentes formatos.

Para poder guardar de forma persistente (por ejemplo, en el disco duro) la información almacenada en memoria tanto en **Objects** como en **ArrayList de Objects** necesitamos realizar un **mapeo** y/o una **serialización**.

Serializar es el proceso de convertir una estructura de datos en memoria (por ejemplo, un objeto, una colección o una estructura compleja) a una secuencia de bytes o texto para poder almacenarla, transmitirla o reconstruirla después.

Mapear (en programación) significa transformar datos de una representación o estructura a otra preservando la correspondencia entre elementos.

Generalmente el mapeo incluye una serialización que en nuestros ejemplos serán a texto.

15.1.- Mapeo con archivos en formato JSON

Como ejemplo, se proporciona un proyecto denominado **Ficheros01JacksonJSON** que muestra los siguientes pasos:

Ficheros01JacksonJSON

```
// ***********************************
// Inicializar lista de Objects

ArrayList<Usuario> usersArrayList = new ArrayList<>();
Usuario usu1 = new Usuario("Pérez, Antonio", 32, "1234");
Usuario usu2 = new Usuario("López, Juan", 28, "abcd");
usersArrayList.add(usu1);
usersArrayList.add(usu2);

System.out.println("Datos cargados en memoria:");
mostrarUsuarios(usersArrayList);









// ***********************************
// JSON - Escribir ArrayList
String jsonList = JacksonMapper.jsonMapperWriter(usersArrayList);
JacksonMapper.writeToFile("./datos/json_usuarios.json", jsonList);

// JSON - Leer ArrayList
jsonList = JacksonMapper.readFromFileDirect("./datos/json_usuarios.json");
ArrayList<Usuario> usuariosArray =
           JacksonMapper.jsonArrayMapperReader(jsonList, Usuario.class);

// Mostrar Datos
System.out.println("");
System.out.println("ArrayList cargado desde archivo JSON:");
mostrarUsuarios(usuariosArray);
```

```
// **********************************
// JSON - Escribir Object
String jsonObj = JacksonMapper.jsonMapperWriter(usu1);
JacksonMapper.writeToFile("./datos/json_usu1.json", jsonObj);

// JSON - Leer Object
jsonObj = JacksonMapper.readFromFileDirect("./datos/json_usu1.json");
Usuario usuObj = JacksonMapper.jsonObjectMapperReader(jsonObj, Usuario.class);

// Mostrar Datos
System.out.println("");
System.out.println("Object cargado desde archivo JSON: ");
System.out.println(usuObj);
```

Como se puede observar, se usan varios métodos de la clase **JacksonMapper** que tienen la siguiente función:

jsonMapperWriter	**Mapeo JSON a String** Obtiene un String en formato JSON a partir de un Object o ArrayList de Objects
jsonArrayMapperReader	**Mapeo JSON desde String** Obtiene un ArrayList de Objects a partir de un String
jsonObjectMapperReader	**Mapeo JSON desde String** Obtiene un Object a partir de un String

writeToFile	Escribe un String en un fichero de texto
readFromFileDirect	Obtiene un String a partir del contenido de un fichero de texto

15.2.- Mapeo con archivos en formato YAML

Se proporciona un proyecto denominado **Ficheros02JacksonYAML** que realiza el mismo proceso que el anterior pero usando formato **YAML** en vez de **JSON.**

YAML (YAML Ain't Markup Language) es un formato de serialización de datos legible por humanos, diseñado para representar estructuras como mapeos (diccionarios), secuencias (listas) y valores escalares (cadenas, números, booleanos, nulos).

YAML respecto a JSON, :

- Es más legible para humanos
- Soporta comentarios y referencias (JSON no).

Conversor de JSON a YAML
https://www.bairesdev.com/tools/json2yaml/

15.3.- Mapeo con archivos en formato CSV

Se proporciona un proyecto denominado **Ficheros03JacksonCSV** que también realiza el proceso de mapeo pero usando formato **CSV.**

El formato **CSV** está más limitado que los anteriores y solo permite escribir ArrayList de Objects ya que mantiene una estructura de tabla similar a una hoja de cálculo.

CSV (Comma Separated Values) es un formato serialización en fichero de texto plano que almacena datos en forma de tabla. Cada línea representa una fila y cada valor está separado por un delimitador, generalmente una coma o punto y coma.

La primera fila suele contener las cabeceras (nombres de las columnas), y las siguientes contienen los datos.

Este formato suele usarse en migración de datos entre diferentes bases de datos.

15.4.- Mapeo con archivos en formato XML

Se proporciona un proyecto denominado **Ficheros04JacksonXML** que también realiza los procesos de mapeo pero usando formato XML**.**

El formato **XML** dispone de etiquetas para definir los datos.
Para poder mapear correctamente, **Jackson** define el contenido de estas etiquetas usando anotaciones en las clases de datos.

En nuestro ejemplo, se han usado las siguientes anotaciones:

@JsonRootName	Nombre de la etiqueta root del XML
@JsonPropertyOrder	Orden de escritura de las propiedades o campos
@JacksonXmlProperty	Nombre de la etiqueta de cada propiedad del XML. También define como se escribe (por ejemplo como atributo o campo)
@JsonIgnore	Campo que no debe escribirse en el fichero